

```

"""
import_json_to_sqlite.py (v2)

Membaca file JSON produk hierarkis (Shipping
Case -> Bundle -> Pack -> ...)
dan menyimpan tiap LEVEL/ProductType ke TABEL
TERSENDIRI, memakai
PrimaryProductKey sebagai primary key & join key
ke parent-nya
(parent_product_key), sesuai schema.sql.

Cara pakai:
    python import_json_to_sqlite.py data.json
products.db
"""

import json
import re
import sqlite3
import sys
import os

# Mapping nama ProductType (dari JSON) -> nama
tabel yang sudah didefinisikan di schema.sql
KNOWN_TYPE_TABLES = {
    "Shipping Case": "shipping_case",
    "Bundle": "bundle",
    "Pack": "pack",
}

def sanitize_table_name(name: str) -> str:
    """Ubah nama ProductType jadi nama tabel
yang valid, mis. 'Master Case' ->

```

```

'master_case'."""
    name = name.strip().lower()
    name = re.sub(r"^[a-z0-9]+", "_", name)
    return name.strip("_")

def ensure_product_table(cur, table_name: str):
    """Buat tabel produk baru dengan struktur
    sama seperti shipping_case/bundle/pack
    jika ProductType belum terdaftar di
    KNOWN_TYPE_TABLES / schema.sql."""
    cur.execute(
        f"""CREATE TABLE IF NOT EXISTS "
{table_name}" (
        primary_product_key    TEXT
PRIMARY KEY,
        parent_product_key    TEXT,
        parent_table          TEXT,
        customer_code         TEXT,
        definition_key         TEXT,
        product_type_name     TEXT,
        quantity              REAL
    )"""
    )
    cur.execute(f'CREATE INDEX IF NOT EXISTS
"idx_{table_name}_parent" ON "{table_name}"
(parent_product_key)')

def get_table_for_type(cur, product_type_name:
str) -> str:
    if product_type_name in KNOWN_TYPE_TABLES:
        return
    KNOWN_TYPE_TABLES[product_type_name]
    table_name =
    sanitize_table_name(product_type_name or
    "unknown_type")

```

```

ensure_product_table(cur, table_name)
return table_name

# -----
-----
# Master / reference data (natural key sebagai
# PK -> pakai INSERT OR IGNORE)
# -----
-----

def upsert_customer(cur, customer):
    if not customer:
        return None
    cur.execute(
        "INSERT OR IGNORE INTO customers (code,
name, owner) VALUES (?, ?, ?)",
        (customer.get("Code"),
customer.get("Name"), customer.get("Owner")),
    )
    return customer.get("Code")

def upsert_brand_family(cur, brand_family):
    """Rekursif: simpan ParentBrandFamily dulu,
baru brand_family itu sendiri."""
    if not brand_family:
        return None

    parent_name = upsert_brand_family(cur,
brand_family.get("ParentBrandFamily"))
    name = brand_family.get("Name")

    cur.execute(
        "INSERT OR IGNORE INTO brand_families
(name, description, parent_name) VALUES (?, ?,
?)",

```

```

        (name, brand_family.get("Description"),
parent_name),
    )
    return name

def upsert_product_definition(cur,
product_definition):
    if not product_definition:
        return None

    definition_key =
product_definition.get("ProductDefinitionKey")
    brand_family_name = upsert_brand_family(cur,
product_definition.get("BrandFamily"))
    category =
product_definition.get("ProductCategory") or {}

    cur.execute(
        """INSERT OR IGNORE INTO
product_definitions
            (definition_key, description,
product_category, brand_family_name)
            VALUES (?, ?, ?, ?)""",
        (
            definition_key,

product_definition.get("Description"),
            category.get("Name"),
            brand_family_name,
        ),
    )

    for prop in
product_definition.get("ProductDefinitionPropert
ies") or []:
        cur.execute(

```

```

        """INSERT OR IGNORE INTO
product_definition_properties
        (definition_key, prop_name,
prop_value) VALUES (?, ?, ?)"""
        (definition_key, prop.get("Name"),
prop.get("Value")),
        )

        return definition_key

# -----
# -----
# Produk (rekursif, per level ProductType masuk
ke tabelnya sendiri)
# -----
# -----

def insert_product(cur, node,
parent_product_key=None, parent_table=None):
    product_type_name = (node.get("ProductType")
or {}).get("Name")
    table_name = get_table_for_type(cur,
product_type_name)

    primary_key = node.get("PrimaryProductKey")
    customer_code = upsert_customer(cur,
node.get("Customer"))
    definition_key =
upsert_product_definition(cur,
node.get("ProductDefinition"))

    cur.execute(
        f"""INSERT OR REPLACE INTO "
{table_name}"
        (primary_product_key,
parent_product_key, parent_table,

```

```

        customer_code, definition_key,
product_type_name, quantity)
        VALUES (?, ?, ?, ?, ?, ?, ?)"""
    (
        primary_key,
        parent_product_key,
        parent_table,
        customer_code,
        definition_key,
        product_type_name,
        node.get("Quantity"),
    ),
)

# ProductProperties (key-value bebas, field-nya bervariasi antar produk)
for prop_name, prop_value in
(node.get("ProductProperties") or {}).items():
    cur.execute(
        """INSERT OR REPLACE INTO
product_properties
        (primary_product_key, prop_name,
prop_value) VALUES (?, ?, ?)""",
        (primary_key, prop_name,
str(prop_value)),
    )

# Units
for unit in node.get("Units") or []:
    cur.execute(
        """INSERT INTO units
(primary_product_key, unit_type_name, quantity,
base_unit)
        VALUES (?, ?, ?, ?)""",
        (primary_key,
unit.get("UnitTypeName"), unit.get("Quantity"),
unit.get("BaseUnit")),
    )

```

```

    )

    # AlternateProductKeys
    for alt in node.get("AlternateProductKeys")
    or []:
        cur.execute(
            """INSERT INTO
alternate_product_keys
(primary_product_key,
product_key, product_key_type, product_key_mode)
VALUES (?, ?, ?, ?)""",
            (primary_key, alt.get("ProductKey"),
alt.get("ProductKeyType"),
alt.get("ProductKeyMode")),
        )

    # Rekursif untuk ChildProducts -> masing-
    masing masuk ke tabel sesuai ProductType-nya,
    # dengan parent_product_key & parent_table
    menunjuk ke produk saat ini.
    for child in node.get("ChildProducts") or
    []:
        insert_product(cur, child,
parent_product_key=primary_key,
parent_table=table_name)

# -----
# -----
# Main
# -----
# -----

def main():
    if len(sys.argv) != 3:
        print("Cara pakai: python
import_json_to_sqlite.py <file_json>

```

```

<file_database.db>")
    sys.exit(1)

    json_path = sys.argv[1]
    db_path = sys.argv[2]

    if not os.path.exists(json_path):
        print(f"File JSON tidak ditemukan:
{json_path}")
        sys.exit(1)

    schema_path =
os.path.join(os.path.dirname(os.path.abspath(__f
ile__)), "schema.sql")

    with open(json_path, "r", encoding="utf-8")
as f:
        data = json.load(f)

    # Jika file berisi list of products, proses
satu per satu.
    # Jika berisi 1 object produk (seperti
contoh), bungkus jadi list.
    if isinstance(data, dict):
        data = [data]

    conn = sqlite3.connect(db_path)
    cur = conn.cursor()
    cur.execute("PRAGMA foreign_keys = ON;")

    with open(schema_path, "r", encoding="utf-
8") as f:
        cur.executescript(f.read())

    try:
        for product_node in data:
            insert_product(cur, product_node)

```

```
        conn.commit()
        print(f"Berhasil! Data dari
'{json_path}' sudah diimport ke '{db_path}'.")
    except Exception as e:
        conn.rollback()
        print(f"Gagal insert, rollback. Error:
{e}")
        raise
    finally:
        conn.close()

if __name__ == "__main__":
    main()
```